Project PRD: Smart Abandoned Cart Notifier (v2)

Overview

This project focuses on a backend-heavy microservice that triggers automated reminders via SMS, WhatsApp, Email, or FCM when a user abandons their cart. The frontend is a minimal React Native application (using TailwindCSS) with 2-3 preset items, quantity controls, a random item adder, and a button to simulate cart abandonment. On abandonment, the backend (Node.js with Express and TypeScript) processes the event, uses Redis to enforce a rate limit of 2 requests per minute, applies an agentic AI chain (using free-tier LLM such as Gemini's free API key) to generate personalized messages, and dispatches the message through a free-tier messaging service.

Key Features

- React Native frontend (TypeScript + TailwindCSS) with:
  - 2-3 pre-added items and quantity (+/-) buttons.
  - 'Add Random Item' button to append a random product.
  - 'Simulate Abandon Cart' button to send the current cart state and user info to backend.
- Node.js + Express (TypeScript) backend endpoints:
  - POST /api/cart/abandon: receives userName, contact (email/phone/whatsapp), and items array.
  - Rate limiting middleware (Redis): max 2 abandonment events per minute per contact.
- Agentic AI chain:
  - Step 1: Prompt LLM (free-tier Gemini or equivalent) to generate a 2-line personalized reminder message using userName and items.
  - Step 2: Prompt LLM again to refine tone and personalization.
  - Ensure consistency by verifying with a quick follow-up prompt ('Anything to improve?').
- Free-tier messaging integrations:
  - Email via SendGrid Free Tier (100 emails/day) or nodemailer with Gmail SMTP.
  - SMS via Twilio Free Trial (sandbox number).
  - WhatsApp via Twilio WhatsApp Sandbox (free limited usage).
  - FCM (if later using mobile app push), Firebase free tier.
- Logging & temporary storage: logs to console or local JSON file to showcase message dispatch.

Tech Stack & Libraries

Frontend (React Native + TypeScript + TailwindCSS):
- React Native CLI or Expo (TypeScript template).
- TailwindCSS via NativeWind or Twin.macro.
- Axios or Fetch for API requests.
- No complex UI libraries: focus on basic Buttons, Text, and View components.

Backend (Node.js + Express + TypeScript):
- Node.js (v18+), Express (v4+), TypeScript (v5+).
- Redis (v6+) for rate limiting (using ioredis).
- Agentic AI integration:

- Axios or node-fetch for HTTP requests to free-tier LLM endpoint (Gemini API).
- Optional: OpenAI free-tier compatible LLM or Hugging Face open-source model if Gemini is not available.
- Messaging libraries:
- nodemailer (for Email via Gmail SMTP or SendGrid).
- twilio-node (for SMS and WhatsApp Sandbox).
- firebase-admin (for FCM, if later).
- Rate-Limiting Middleware:
- Custom Express middleware using Redis GET/INCR/EXPIRE logic for 2 requests/minute.
- Type Safety:
- zod or io-ts for runtime validation of incoming payloads.
- TypeScript interfaces for request/response DTOs.

Development Flow

1. Frontend Setup (React Native):
- Initialize a new React Native project (TypeScript template).
- Install NativeWind for Tailwind support.
- Hardcode an array of 2-3 products with fields { id, name, quantity } in state.
- Render each product with Text components and +/- Buttons to modify quantity.
- 'Add Random Item' Button: onPress generates a random product (e.g., random from a small list) and appends to the array.
- 'Simulate Abandon Cart' Button: onPress sends a POST to backend:

```
{
  userName: string,
  contact: { email?: string; phone?: string; whatsapp?: string },
  items: Array<{ id: string; name: string; quantity: number }>
}
```

2. Backend Setup (Node.js + Express):
- Initialize a new Node.js project. Install express, typescript, ts-node-dev, axios, ioredis, twilio, nodemailer, zod.
- Create an Express app (app.ts) with JSON body parsing.
- Implement Rate-Limiting Middleware:

```ts
import { Request, Response, NextFunction } from 'express';
import Redis from 'ioredis';
const redis = new Redis();
export async function rateLimit(req: Request, res: Response, next: NextFunction) {
  const contact = req.body.contact.phone || req.body.contact.email || req.body.contact.whatsapp;
  const key = `rate:${contact}`;
  const count = await redis.get(key);
  if (count && parseInt(count) >= 2) {
    return res.status(429).json({ message: 'Too many requests. Try again later.' });
```

```
    }
    const pipeline = redis.pipeline();
    pipeline.incr(key);
    pipeline.expire(key, 60); // 2 requests per 60 seconds
    await pipeline.exec();
    next();
  }
  ```
- Register middleware globally: app.use(rateLimit); before routes.
- Create Route Handler POST /api/cart/abandon:
  ```ts
  import { z } from 'zod';
  const CartEventSchema = z.object({
    userName: z.string(),
    contact: z.object({
      email: z.string().email().optional(),
      phone: z.string().optional(),
      whatsapp: z.string().optional(),
    }),
    items: z.array(z.object({ id: z.string(), name: z.string(), quantity: z.number() })),
  });
  app.post('/api/cart/abandon', async (req, res) => {
    const parseResult = CartEventSchema.safeParse(req.body);
    if (!parseResult.success) {
      return res.status(400).json({ error: parseResult.error.format() });
    }
    const { userName, contact, items } = parseResult.data;
    // Agentic AI:
    // 1. Call LLM (Gemini free API) to generate initial 2-line message.
     const initialPrompt = `Generate a 2-line personalized reminder for ${userName} who left ${items.length} items in cart: ${items.map(i => i.name).join(', ')}.`;
     const initialResponse = await axios.post('https://gemini-api-free.example/v1/chat', { prompt: initialPrompt });
    const initialText = initialResponse.data.choices[0].text;
    // 2. Refinement Prompt:
    const refinePrompt = `Improve personalization and add a friendly tone to: ${initialText}`;
    const refineResponse = await axios.post('https://gemini-api-free.example/v1/chat', { prompt: refinePrompt });
    const finalMessage = refineResponse.data.choices[0].text;
    // 3. Dispatch Message via Free-Tier Service:
    try {
      if (contact.email) {
```

```
      // Send via nodemailer using Gmail SMTP or SendGrid
    } else if (contact.phone) {
      // Send via Twilio SMS (free trial)
    } else if (contact.whatsapp) {
      // Send via Twilio WhatsApp Sandbox
    }
    // Log dispatch
    console.log(`Sent message to ${JSON.stringify(contact)}: ${finalMessage}`);
    res.status(200).json({ success: true, message: finalMessage });
  } catch (err) {
    res.status(500).json({ error: 'Failed to send message.' });
  }
});
```

- Start server with ts-node-dev app.ts.

3. Agentic AI Configuration:
   - Sign up for Google Cloud or Hugging Face for free-tier LLM access (Gemini or open-source).
   - Store API key in environment variable (e.g., GEMINI_API_KEY).
   - Use axios to call endpoint, handle rate-limit on API.

4. Messaging Integration:
   - Email: configure nodemailer with Gmail SMTP (set up App Password) or SendGrid free API key.
   - SMS: create a Twilio free trial account; use the sandbox phone number (no cost for limited messages).
   - WhatsApp: configure Twilio WhatsApp Sandbox (free usage, limited to approved numbers).

5. Testing & Demo:
   - Run the React Native app on emulator or device.
   - Use hardcoded userName and contact (e.g., an email you control or sandbox phone).
   - Modify quantities, click 'Simulate Abandon Cart', observe console logs and message reception.
   - Verify rate limiting by clicking twice quickly and seeing 429 response.

6. Future Enhancements (Post-MVP):
   - Integrate MongoDB to persist user flows and notifications.
   - Add Clerk authentication and user management.
   - Build a simple dashboard to view sent messages and analytics.
   - Expand channel support (push notifications via FCM for mobile).